RailRoute Lab

One problem. One inspectable AI core. Reproducible evidence.

Problem: Find the minimum modeled shuttle travel-time route between two campus locations for a selected departure time and tunnel-closure state.

- A fictional 14-location, 17-road network with a railway barrier and two tunnels.
- A* performs the route search locally in Python; the 3D view displays its result.
- Outputs: route, travel time, road costs, and every A* expansion decision.

Model boundary: generated road times and congestion; no measured VIT traffic or live ETA.

Formal search formulation

What the algorithm is allowed to do

- **State:** current road location and arrival time.
- **Action:** traverse an adjacent open road.
- **Transition:** reach its neighbor after its entry-time travel cost.
- **Goal:** current location equals destination.
- **Cost:** sum of modeled driving seconds.

Only two graph edges cross the railway. Closing both makes cross-rail trips unreachable.

Inside A*

The exact decision rule used in routing.py

f(n) = g(n) + h(n)

- **g:** modeled travel time already spent from the start.
- **h:** straight-line distance to goal ÷ 15 km/h; a safe lower bound.
- **f:** priority in the waiting queue; the lowest f is expanded next.
- A better arrival updates best_g and a parent link; parents reconstruct the route.

The heuristic is admissible because roads are at least straight-line length and never exceed the assumed maximum speed.

A live route decision

Seed 17: A to D

12:00, both open	WEST	8.30 min
08:30, both open	EAST	12.46 min
12:00, west closed	EAST	9.17 min
12:00, both closed	NO ROUTE	queue exhausted

Show the expansion list and road decisions; the shuttle animation is only a playback of the chosen route.

Comparison evidence

A* versus Dijkstra (same search engine, h = 0)

23,520 test cases • **20,640** reachable • **0** cost/reachability disagreements

- A* expanded fewer locations in 13,647 reachable cases; equal in 6,993.
- Cost agreement is a consistency check, not independent proof: the methods share code.

Demonstrate and defend

What to run and what to explain in the viva

- Run **python review_demo.py**; change time, start, destination, seed, and closures.
- Point to **routing.py / _search**: pop lowest f, evaluate roads, update best_g, save parent.
- Explain why h is a lower bound and why arrival time affects road cost.
- Show no-route output when both tunnel edges are closed.
- State the limits: synthetic map, generated traffic, and no passenger waiting time.

Disclosure: AI assisted with code and presentation drafting. The local Python algorithm makes every route decision. Each teammate must verify and explain it.